

Software, en general, es un conjunto de programas y la documentación que acompaña.

Hay tres tipos de software principales

- Software de sistema

Es el que administra y controla los recursos de hardware de la computadora. Permite que los programas se ejecuten y actúen como la interfaz entre el hardware y el software de aplicación. Por ejemplo sistemas operativos, controladores de dispositivos.

- Software de utilidades

Ayuda a mantener y optimizar el rendimiento de la computadora, realizando tareas de mantenimiento o gestión. Funciona como una herramienta para el sistema operativo o el hardware. Por ejemplo antivirus, gestor de disco o compresores de archivos.

- Software de aplicación

Diseñado para realizar tareas específicas para el usuario final. Está construido sobre el software del sistema y necesita un sistema operativo para poder ejecutarse. Por ejemplo procesadores de texto, navegadores web, videojuegos, etc.

El software no se debe comparar con la manufactura:

- La manufactura sigue un plan fijo, mientras que el desarrollo de software es un proceso flexible donde los requisitos pueden cambiar.
- En la manufactura se hacen productos idénticos en grandes cantidades. El software, en cambio, se personaliza y actualiza constantemente, haciendo que casi cada versión sea única.
- Los fallos en la manufactura se deben a errores físicos. En el software, las fallas pueden ser causadas por problemas de compatibilidad o entorno, no siempre por un error de código.
- Los productos manufacturados se degradan con el tiempo y el uso. El software es inmaterial y no se "gasta" por más que se use.
- La manufactura está sujeta a las leyes de la física (gravedad, materiales, etc.). El software, al ser una creación lógica, se rige por sus propias reglas.

Existen tres disciplinas:

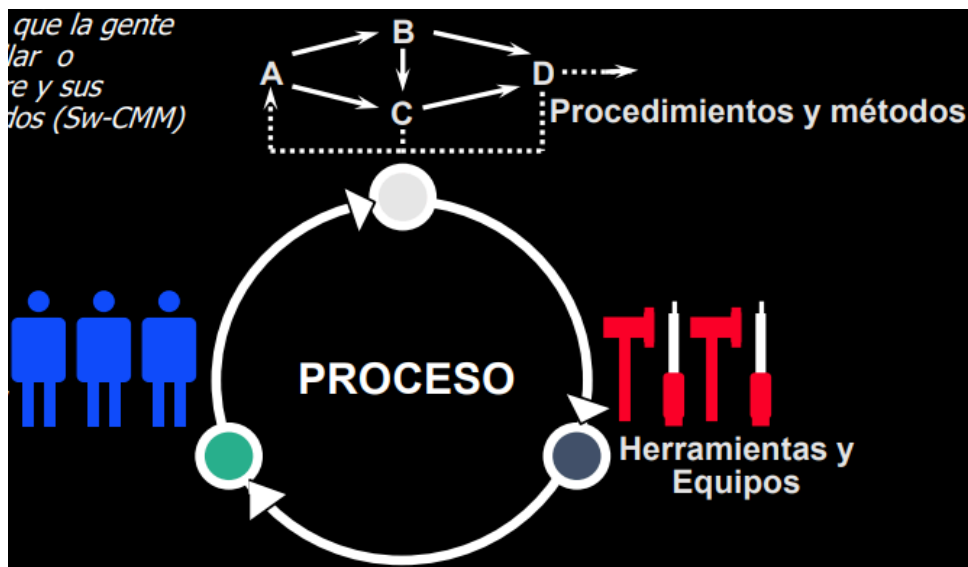
- Técnicas
 - Requerimientos, análisis y diseño, construcción, prueba y despliegue.
- Gestion
 - Planificación del proyecto y monitoreo y control del proyecto.
- Soporte
 - Gestión de configuración de software, aseguramiento de calidad y métricas.

Procesos de Software



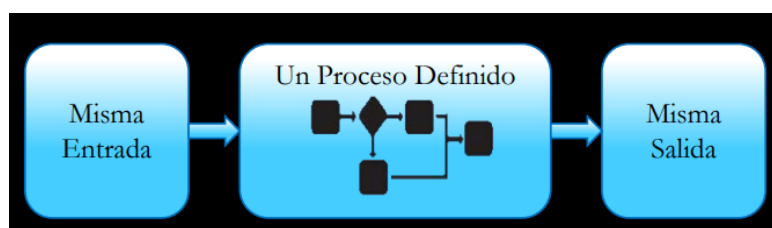
Conjunto estructurado de actividades para desarrollar un sistema de software. Estas actividades varían dependiendo de la organización y el tipo de sistema que debe desarrollarse. Debe ser explícitamente modelado si va a ser administrado.

Un proceso es una secuencia de pasos ejecutados para un propósito dado. Y un proceso de software es un conjunto de actividades, métodos y prácticas que las personas usan para desarrollar o mantener software. Este proceso es cíclico y depende de tres elementos clave.



Procesos Definidos y Procesos Empíricos

Un **proceso definido** es una serie de pasos repetitivos que se inspira en el modelo de las líneas de producción de manufactura. La idea central es que, si se sigue el mismo proceso con la misma entrada, se obtendrá siempre la misma salida.



- **Predecibilidad:** El proceso es diseñado para ser predecible..
- **Repetibilidad:** Se asume que la secuencia se puede repetir para lograr resultados consistentes.
- **Enfoque en la documentación:** Los "Entregables/Hitos" y las "Revisiones/Aprobaciones" en cada etapa. Clave de un flujo de trabajo controlado en un proceso definido.

Los **procesos empíricos** se basan en la experiencia y la adaptación, asumen que cada ejecución es única debido a las variables cambiantes.

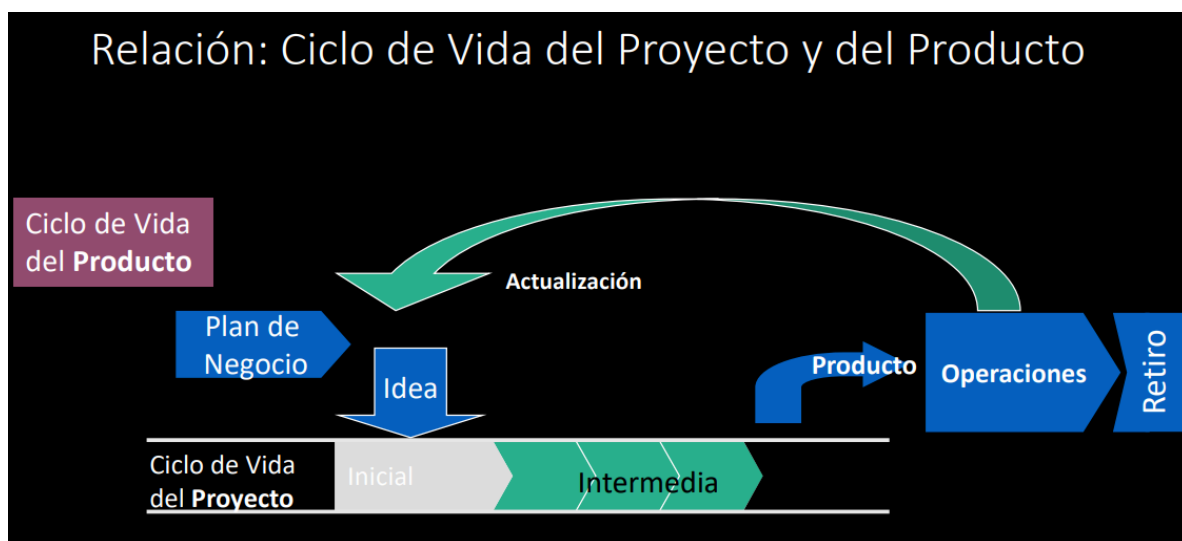
- **Adaptabilidad:** Asumen que el proceso es complicado y que las variables pueden cambiar. Por lo tanto, el resultado puede ser diferente cada vez que se repite.
- **Control por Inspección y Adaptación:** La gestión de estos procesos se basa en la observación constante de los resultados (**inspección**) y en la capacidad de ajustar el plan de acción (**adaptación**).
- **Procesos Creativos:** El desarrollo de software, al ser un proceso que a menudo requiere innovación y resolución de problemas, se beneficia de este enfoque.

Suena a metodologías ágiles como Scrum o Kanban donde la planificación y la ejecución son iterativas y se adaptan a los cambios y necesidades. Su proceso es similar a :

1. **Asumir:** Se parte de un conocimiento inicial o una suposición.
2. **Construir:** Se desarrolla parte del producto basada en las suposiciones iniciales.
3. **Retroalimentar:** Se obtiene feedback sobre el trabajo realizado.
4. **Revisar:** Se analiza la retroalimentación para entender qué funcionó y qué no.
5. **Adaptar:** Se ajusta el plan, el producto para el siguiente ciclo. Lo que se aprendió se usa para tomar mejores decisiones en el futuro.

Ciclos de vida

La serie de pasos a través de los cuales el producto o proyecto progresa. Los productos tienen su ciclo de vida al igual que los proyectos.



El ciclo de vida del proyecto es un proceso temporal con un principio y un final definidos. Se enfoca en la creación de un resultado único.

- **Fase inicial:** Comienza con el plan de negocio. Es la etapa de planificación donde se decide qué se va a construir.
- **Fase intermedia:** El proyecto avanza a través de las etapas de desarrollo (diseño, construcción, pruebas) para entregar el producto.
- **Fin del proyecto:** Un proyecto termina cuando el producto es entregado.

Fases Comunes

- **Requerimientos:** Se definen y analizan las necesidades y expectativas del cliente para el software.
- **Diseño:** Se planifica la arquitectura del software, los componentes, las interfaces y la base de datos.
- **Implementación:** El código fuente del software se escribe.
- **Pruebas:** Se verifican el software para detectar y corregir errores, asegurando que cumpla con los requisitos.
- **Despliegue:** El software se lanza y se pone a disposición de los usuarios finales.
- **Mantenimiento:** El software se corrige, actualiza y mejora continuamente después de su lanzamiento.

Modelos Comunes de Ciclo de Vida

- **Modelo en Cascada:** Es un modelo lineal y secuencial. Cada fase debe completarse completamente antes de pasar a la siguiente.
- **Modelo Ágil:** Se centra en el desarrollo iterativo e incremental, con ciclos cortos (sprints). La retroalimentación y la adaptación son la clave.
- **Modelo en Espiral:** Combina el enfoque en cascada con el iterativo. Cada espiral o es una vuelta que se centra en la gestión de riesgos en cada fase.

El ciclo de vida del producto es un proceso continuo que dura mucho más. Comienza cuando el producto es concebido y termina cuando es retirado del mercado.

- **Inició:** El producto nace de la fase de proyecto, cuando es entregado.
- **Operaciones:** Una vez lanzado, el producto entra en la fase de **Operaciones**. Se gestiona, se mantiene y se ofrece a los usuarios finales.
- **Actualización:** Son los "proyectos de actualización", para mejorar o modificar el producto existente. Esto es lo que permite que el producto se mantenga relevante.
- **Retiro:** El ciclo de vida del producto finaliza cuando se decide que ya no es viable y es retirado.

Clasificación de los ciclos de vida

- **Secuencial**

Cada fase del proyecto debe completarse antes de pasar a la siguiente. Es un enfoque muy estructurado, ideal para proyectos con requerimientos bien definidos. La principal desventaja es su falta de flexibilidad para adaptarse a cambios una vez que el proyecto ha avanzado.

- **Iterativo/Incremental**

El software se construye en múltiples iteraciones. El proyecto se divide en mini-proyectos más pequeños, y en cada ciclo se construye una parte funcional del producto. Este tipo de ciclo de vida es excelente para proyectos donde los requerimientos son cambiantes, ya que permite la retroalimentación constante y la adaptación.

- **Recurso**

Se centra en la gestión de riesgos y la evolución del proyecto. Cada vuelta en la espiral representa una fase del proyecto, que avanza a través de una serie de pasos que incluyen la planificación, el análisis, la ingeniería y la evaluación. Este modelo es más complejo, pero es ideal para proyectos grandes y de alto riesgo, ya que permite identificar y mitigar los riesgos de manera temprana y continua.

La relación entre un **proceso de desarrollo** y un **ciclo de vida** de software es que el ciclo de vida es una representación del proceso. El proceso es la forma en que se realiza el trabajo, y el ciclo de vida es el modelo que describe cómo se organizan y gestionan las actividades de ese proceso.

Software Configuration Management (SCM)



Elementos Clave

- **Personas**: Son los individuos que conforman el equipo de desarrollo
- **Proyecto**: Un proyecto obtiene como resultado un producto.
- **Producto**: Es el software final que se crea.
- **Proceso**: Es el conjunto de actividades que el equipo sigue para ejecutar el proyecto.
- **Herramientas**: Son los programas, equipos y tecnologías que se utilizan para automatizar y facilitar las actividades del proceso.

Se nota una interdependencia ya que las personas trabajan en un proyecto, siguiendo un proceso que se adapta a las necesidades del proyecto. Las herramientas se utilizan para

automatizar y optimizar ese proceso, y el resultado de todo el esfuerzo es el producto de software.

Los cambios en el software son inevitables, pueden ser del tipo: cambios impulsados por el negocio y los requerimientos

- Cambios del negocio y nuevos requerimientos
- Reorganización de las prioridades de la empresa
- Cambios en servicios asociados
- Cambios en el presupuesto

o cambios impulsados por la calidad y la mejora continua

- Defectos encontrados a corregir
- Oportunidades de mejora

La Gestión de la Configuración del Software (SCM) es una disciplina definida como una actividad "paraguas", es transversal a todo el proyecto y es relevante a lo largo de todo el ciclo de vida del producto.

Se encarga de gestionar y controlar los cambios en los artefactos del software. Estos artefactos no son solo el código, sino también la documentación, los requerimientos, los diseños y los datos de prueba.

Las principales funciones del SCM son:

- Identificación de la configuración: Definir y etiquetar los elementos de software para controlarlos y rastrearlos de manera única.
- Control de cambios: Gestionar y aprobar los cambios en los artefactos de forma sistemática para evitar errores y caos.
- Gestión de versiones: Crear y mantener versiones de los productos de software, permitiendo a los equipos volver a versiones anteriores si es necesario.
- Auditoría de la configuración: Asegurar que el estado de la configuración es el esperado y que se cumplen los requisitos.
- Informes de estado: Proporcionar información sobre el estado actual de los elementos y los cambios realizados.

El SCM no es una fase más del proyecto, sino un conjunto de prácticas que se aplican para garantizar que el desarrollo sea ordenado y que la calidad del producto se mantenga.

La definición que da la ANSI/IEEE 828, 1990 , es una disciplina que aplica dirección y monitoreo administrativo y técnico para

- identificar y documentar las características funcionales y técnicas de los ítems de configuración
- controlar los cambios de esas características (de arriba)
- registrar y reportar los cambios y su estado de implementación
- verificar correspondencia de los cambios con los requerimientos

¿Por qué deberíamos gestionar la configuración del software?

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida

1. Controlar los cambios

El software está en constante evolución. Sin un control sistemático, los cambios pueden introducir errores o incompatibilidades. El SCM asegura que cada cambio se apruebe, se documente y se integre de manera controlada.

2. Mantener la integridad y la consistencia

El SCM permite mantener la integridad de la base de código. Se asegura de que todas las versiones del software sean consistentes y correctas. Esto garantiza que todos los miembros del equipo trabajen en la misma versión del código

3. Trazabilidad y origen

Con SCM, cada cambio en el software queda registrado. Es posible rastrear quién hizo un cambio, cuándo y por qué. Esto es crucial para la auditoría, la depuración de errores y para entender el historial del desarrollo.

Integridad del producto

1. Satisfacción de las Necesidades del Usuario

La integridad del software se mide por su capacidad para satisfacer las necesidades y los requisitos para los que fue diseñado.

2. Trazabilidad Completa

Un producto íntegro debe poder ser rastreado a lo largo de su ciclo de vida.

3. Cumplimiento de los Criterios de Rendimiento

La integridad de un producto también se basa en su rendimiento. Debe operar de manera eficiente, ser confiable y comportarse de la forma esperada

4. Cumplimiento de las Expectativas de Costo

Un producto con integridad cumple con las expectativas de costo establecidas en su plan de negocio.

Problemas con el manejo de componentes

1. Pérdida de un componente

Si no se gestiona adecuadamente, un componente puede perderse durante las transferencias entre equipos.

2. Pérdida de cambios

Esto ocurre cuando se trabaja con una versión obsoleta de un componente. Si un desarrollador edita una versión antigua, sus cambios sobrescribirán las modificaciones más recientes lo que resulta en la pérdida de trabajo.

3. Sincronía entre la fuente, el objeto y el ejecutable

El código fuente debe coincidir con el código compilado y el archivo ejecutable final. Si un cambio en el código fuente no se compila correctamente puede generar errores.

4. Regresión de fallas

Una regresión ocurre cuando un cambio causa que una funcionalidad que antes funcionaba deje de hacerlo. Esto a menudo sucede cuando un cambio en un componente afecta a otro componente sin tener el cuidado correspondiente.

5. Doble mantenimiento

Esto sucede cuando dos o más desarrolladores arreglan el mismo problema de manera independiente.

6. Superposición de cambios overlap

Cuando dos o más personas trabajan en el mismo componente al mismo tiempo, sus cambios pueden sobrescribirse mutuamente

7. Cambios no validados

Los cambios se aplican al código base sin haber pasado por un proceso de revisión. Esto aumenta el riesgo de introducir nuevos errores.

Conceptos claves

- Ítem de Configuración

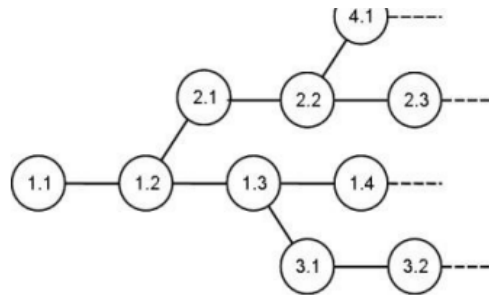
Se llama ítem de configuración (IC) a todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución. Por ejemplo, Propuestas de Cambio, Plan de desarrollo, Manual de Usuario, Requerimientos, Casos de prueba, Código fuente, etc

- Versión

Una versión se define como la forma particular de un artefacto en un instante dado. El control de versiones se refiere a la evolución de un único ítem de configuración (IC), o de cada IC por separado. La evolución puede representarse gráficamente en forma de grafo.

- Variante

Una variante es una versión de un ítem de configuración que evoluciona por separado. Las variantes representan configuraciones alternativas. Un producto de software puede adoptar distintas configuraciones dependiendo del lugar donde se instale. Por ejemplo, dependiendo de la plataforma que la soporta, o de las funciones opcionales que haya de realizar o no.



- Repositorio

Un repositorio es un lugar de información que contiene los ítems de configuración. Mantiene la historia de cada IC con sus atributos y relaciones. Se usa para hacer evaluaciones de impacto de los cambios propuestos. Pueden ser una o varias bases de datos.

El desarrollador extrae los archivos del repositorio central para trabajar en ellos en su entorno local. Esta acción crea una copia de los archivos en la máquina, lo que le permite hacer cambios de forma segura sin afectar la versión principal del proyecto.

Una vez que se modificaron los archivos, los devuelve al repositorio. Esto actualiza la versión en el repositorio central.

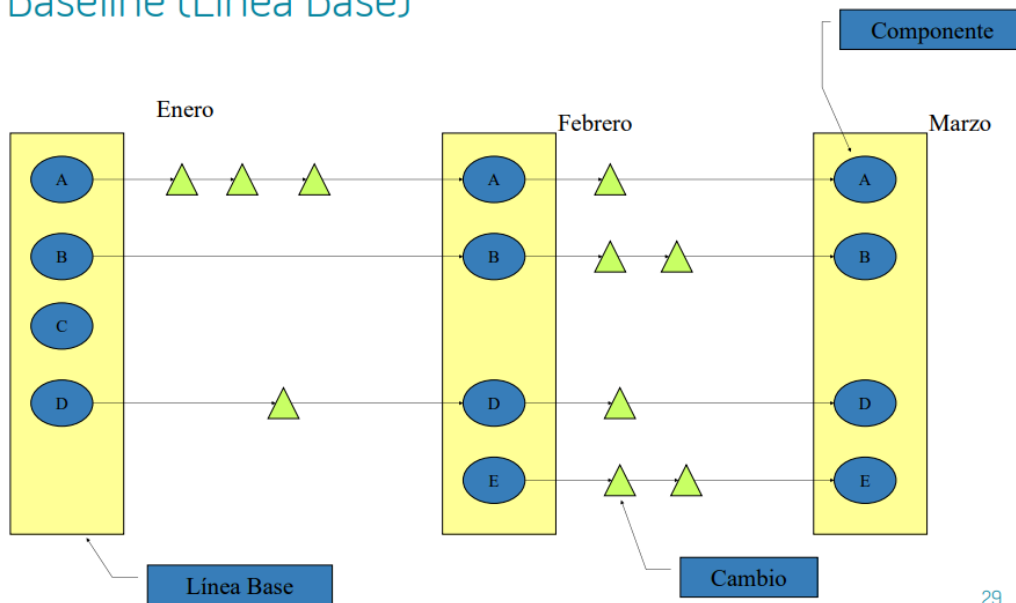
Existen los repositorios centralizados, donde un servidor contiene todos los archivos con sus versiones. Los administradores tienen mayor control sobre el repositorio. Falla el servidor y "estamos al horno". También están los repositorios descentralizados donde cada cliente tiene una copia igual del repositorio completo. Si un servidor falla sólo es cuestión de "copiar y pegar". Posibilita otros workflows no disponibles en el modelo centralizado.

- Línea Base

Una configuración que ha sido revisada y sobre la que se ha llegado a un acuerdo sirve como base para desarrollos posteriores y puede cambiarse sólo a través de un procedimiento formal de control de cambios. Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto

Es una versión estable y aprobada de un conjunto de ítems de configuración en un momento determinado. No hay que confundirla con la versión del producto que es la versión comercial y completa de un software que está disponible para los usuarios, el otro es un punto de referencia interno.

Baseline (Línea Base)



29

Los óvalos azules representan los ítems de configuración.

Los rectángulos amarillos son las líneas base. Por ejemplo, en enero, la línea base del proyecto está compuesta por los componentes A, B, C y D.

Los triángulos verdes representan los cambios. Entre enero y febrero, se realizaron cambios en el componente A y D. La línea base de febrero incluye los componentes A, B, D y un nuevo componente E, además el componente C ya no forma parte de la línea base de febrero. Esto muestra cómo el producto cambia con el tiempo

Tipos de Línea Base:

De especificación → se establecen al final de las fases de planificación y diseño.

Representan la aprobación de los documentos del proyecto, como los requisitos y el diseño arquitectónico. Esto asegura que el equipo tenga una visión clara y acordada del software antes de comenzar a codificar.

De productos → se crean en las fases de desarrollo posteriores a la codificación. Marcan la versión de un producto que ha pasado por un control de calidad y está listo para la siguiente etapa, como las pruebas.

Ramas (Branch) 🌿

Una rama es una copia de la línea principal de desarrollo del código (llamada **main** o **master**). Se utiliza para que los desarrolladores trabajen en nuevas funciones o arreglen errores sin afectar el código base del proyecto. Una vez que el trabajo en la rama está completo y aprobado, se fusiona de nuevo en la línea principal.

Un conjunto de ítems de configuración con su correspondiente versión en un momento determinado.